JUNGHOME_Gateway – Interface Help

JVP process connection for the JUNG HOME Gateway REST API (api-junghome) – including a local HTTP bridge for third-party systems.

1. Overview

This interface connects JUNG Visu Pro to the local JUNG HOME Gateway (api-junghome) over HTTPS. It maps the gateway's Functions (physical devices), their Datapoints (switching/status values) and its Scenes to JVP datapoints. It also optionally provides its own local HTTP bridge, letting third-party systems (e.g. Home Assistant) talk to JVP directly, without their own gateway pairing.

- Read/write Datapoint values (switching, dimming, position, ...)
- Activate Scenes
- Automatic sync (Resync) of Functions/Scenes/Groups
- Local REST bridge (port 8151) in the same JSON format as api-junghome

2. Pairing

- In **CONFIGURATION**: enter the gateway address (HOST), port (default 443) and protocol (HTTPS, default).
- Press the button on the JUNG HOME Gateway, then set **START_PAIRING** to 1.
- **STATUS.PAIRING** shows “waiting for button press”; on success **STATUS.CONNECTED** switches to “connected”.
- The API token is stored automatically in **CONFIGURATION.TOKEN** and used for all subsequent requests.
- If pairing fails (timeout, ~60 attempts at 1s each): press the button again and set START_PAIRING to 1 again.

3. Finding device and scene IDs

- Set **STATUS.RESYNC** to 1 to re-fetch Functions/Scenes/Groups from the gateway.
- **STATUS.FUNCTIONS_JSON** contains all Functions including their Datapoints (id/type/label) as raw data.
- **STATUS.SCENES_JSON** contains all Scenes (id/label).
- The same data is also written in full to the JVP trace log (easier to browse with many devices).

4. Creating instances in the device editor

Instances are created manually (right-click the folder → new instance). The script does not create them itself.

Folder	Instance for	ID field
Function	every physical device (once per device)	FUNCTION_ID
→ Datapoint (nested)	every Datapoint of that Function you want to use	DATAPOINT_ID
Scene	every scene	SCENE_ID

Example: a DimmerLight has two Datapoints (switch, brightness) → one Function instance with two nested Datapoint instances underneath it. Once FUNCTION_ID/DATAPOINT_ID is entered, the instance is loaded live once automatically (label, type, current values). The name reported by the gateway is also written directly into the instance's own folder name.

Each Datapoint can have up to 6 values (VALUE1…VALUE6, each with a matching VALUEn_KEY, e.g. “on_off”, “brightness”, “position”). VALUEn_KEY is filled in automatically during sync; VALUEn is the actual

switch/read value as raw text.

5. Testing

- Write to a **VALUEn** datapoint → forwarded to the gateway immediately via HTTP PATCH.
- Set **ACTIVATE** on a Scene instance to 1 → activates the scene on the gateway.
- A manual “Read” on a datapoint in the editor first live-refreshes the owning instance before answering.
- Automatic background refresh: all configured Datapoint instances are re-read round-robin every 60 seconds (**FUNCTION_REFRESH_INTERVAL_S**), so external changes on the gateway arrive in JVP.
- **STATUS.ERROR**/**LAST_ERROR** show the last error; details are always in the trace log.

6. Local HTTP bridge (optional)

In addition to connecting to the gateway, the interface provides its own small REST server (default port **8151**, see RESOURCES/HTTPSERVER in InterfaceDescription.xml) that mirrors the already synced Function/Datapoint/Scene instances in the same JSON format as api-junghome.

Method	Path	Purpose
GET	/api/junghome/functions/	List of all configured Functions
GET	/api/junghome/functions/{id}	A single Function incl. its Datapoints
GET	/api/junghome/functions/{id}/datapoints/{id}	A single Datapoint with current values
POST	/api/junghome/functions/{id}/datapoints/{id}	Write a value, body {"data":[{"key":..,"value":..}]}
GET	/api/junghome/scenes/	List of all configured Scenes
POST	/api/junghome/scenes/{id}	Activate a scene

Security: set **BRIDGE.TOKEN** to require every request to send the header “Authorization: Bearer <TOKEN>” or “token: <TOKEN>”. Left empty = no check, anyone on the local network can use the bridge. **BRIDGE.REQUEST_COUNT** and **BRIDGE.LAST_REQUEST** show the bridge's usage.

7. Troubleshooting

Symptom	Cause / solution
Connection stays “disconnected”	Wrong HOST/PORT, check the HTTPS setting, or no token (repeat pairing).
Pairing fails / times out	Button on the gateway not pressed in time – press it again, then set START_PAIRING to 1 again.
Function/Datapoint/Scene stays “not found”	Wrong or empty FUNCTION_ID/DATAPOINT_ID/SCENE_ID – copy the values from STATUS.FUNC
Writing fails	FUNCTION_ID/DATAPOINT_ID/key unknown – refresh the instance via RESYNC first; details in LA
Manual “Read” shows a stale value	The interface live-refreshes automatically on an explicit read – if the problem persists, check the trac
HTTP bridge does not respond	Port 8151 blocked by a firewall, or a wrong/missing bridge token in the request header.
SCRIPTNAME error while loading	An old version of the interface is cached in the editor – close and reopen the editor.

Diagnostics: every HTTP request (request and response/error), pairing attempts, resync steps and every incoming value change are written to the JVP trace log – that's where every failure case can be traced in detail.